

Setting up a coding agent

Day-0 baseline or wait for a signal — six decision points in one running case

04

"What in an agent's configuration is worth putting in place right away, on day 0, as a baseline practice — and what cannot be known in advance, because it is specific to your project?"

EMPTY DIRECTORY → SEVEN ADDITIONS, STILL UNNAMED



The real 7 stages of the signup-landing/ demo repository (git ls-tree by commit) · source: captures/21–27

SECTION 0 · THE CLIENT'S REQUEST

The task arrives in the client's own wording: what the result should be is there; how the code is put together, what counts as done, and where the boundaries of action lie are not

"We need a landing page with a sign-up form for a demo lesson: a single static page, the fields 'name' and 'email', a submit button. We have no backend of our own — send the submission to our form-intake service."

"Do we hand this wording to the agent as is — or do we pin down the requirements first?"

open discussion, not a vote

SECTION 0 · SPEC.MD

The requirements break down into functional and non-functional ones and go into spec.md in the first commit; README describes how the repository is put together and is set up in the same commit

```
# spec.md – signup-landing

## Functional requirements
- A static landing page, a sign-up form for a demo lesson
- Fields: name (required), email (required)
- A submit button; no backend – the submission goes to an external service

## Non-functional requirements
- Acceptance criterion: the form actually delivers the submission – verified in prod, not just "the test is green"
- Boundary of action: do not deploy to prod without an explicit request
```

an illustrative example, not a captured artifact

"README.md is set up in the same commit, separately from the spec. The spec is the requirements on the product. The README is how the repository is put together, for whoever walks into it: a human or an agent."

SECTION 0 · THE RUNNING CASE

The agent built a working prototype from the spec without a single clarifying question — and chose the structure of the repository itself along the way, because spec.md does not describe it

```
$ ls -la signup-landing/
total 8
drwxr-xr-x 2 harness harness 4096 Sep 20 12:45 .
drwxr-xr-x 3 harness harness 4096 Sep 20 12:45 ..
```

real output, not a mock

"build a landing page with a sign-up form for a demo lesson: a static HTML page, a Vite build (npm run build → dist), form fields — name and email, a Playwright test that checks that the form does not submit with empty required fields (tests/form.spec.ts)" — matches spec.md in content

"No clarifying question was asked. All the parameters of the task were unambiguously derivable from the text of the task, so work was begun immediately, without clarifications." — a line from the captured session

created by the agent

```
index.html
src/main.js
package.json
tests/form.spec.ts
```

from the final answer

```
npm install
npm run build      # → dist/
npm run test:e2e   # playwright test
```

When setting an agent up in a new repository, everyone works in one of two modes — put the baseline in place right away, or wait for a real problem

A quick read of the room before we go further.

"When you set a new agent up in a new repository — which of the two modes is usually closer to you?"

A · "I put the baseline in place right away"

readiness gate ("what counts as done")

tracking of tasks and decisions

B · "I wait until it hurts"

repository overview

nested convention files

complex, multi-level memory

the breakdown is on the next slide

SECTION 0 · BREAKDOWN OF THE TWO MODES

Both modes work — the question is which type of decision they are applied to: what is specific to the project waits for a signal, universal hygiene is set up right away

SPECIFIC TO THE PROJECT — WAIT FOR A SIGNAL

repository overview

nested convention files

complex, multi-level memory

UNIVERSAL HYGIENE — DAY 0

readiness gate

tracking of tasks and decisions

"Lecture 3 was about a thin agent — about refusing structural complexity: subagents, skills, external access. There a signal is required. Universal hygiene is a different category: here 'wait until it hurts' works against you."

Besides the spec and the README, three more things go into the repository the same day — not one of them waited for a signal

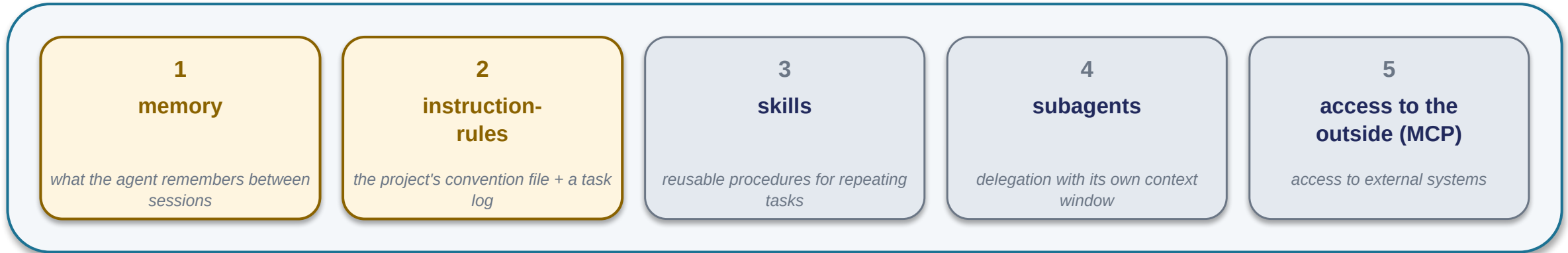
What is set up	Why precisely right away
spec.md + README.md	what the result should be and how the repository is put together — in the first commit, before the task goes to the agent
CLAUDE.md	the instruction file for the agent. On the first day it holds one line: the "done" criterion and a manual check. No description of how the repository is put together — no reason yet
DECISIONS.md	the project's decision log: what was decided and why. So that a rejected option does not have to be worked through again a month later
The file-per-task practice	for a multi-step task — a separate file with the plan and notes on the progress of the work

Each of these costs one line today. The cost of not having it cannot be known in advance and is discovered after the fact: the signal that would have made it clear that it is time may never come at all.

From here on all four are met by name in the work — each in its own decision point.

SECTION 0 · THE HARNESS MAP

An agent's configuration breaks down into five slots; today's project puts two of them to work — instruction-rules and memory



Two are in play today. Skills, subagents, access to the outside and the mechanical layers (hook, process) — Seminar 5.

"Stay on the lowest rung that closes the requirements of the task. Climb to the next one only when you can name a requirement that the current rung does not close. Every climb is paid for with new cost, new failure modes, and a new attack surface."

1

Section 1 · the instruction file

- Case 1.1 · the agent's role and the repository
- Case 1.2 · the basic processes: gates and the improvement loop
- Case 1.3 · nested files per subfolder

ONE FILE, THREE DECISION POINTS



Case 1.1 · the agent's role and the repository

What an empty instruction file gets filled with

THE FIRST CONFIGURATION DECISION



CASE 1.1 · ROLE AND REPOSITORY

The prototype is built, the next session is tomorrow — CLAUDE.md already has the readiness-gate line in it and nothing else, and the standard first section of such a file is called "Repository overview"

repository state

README.md

spec.md

DECISIONS.md

index.html

package.json

src/main.js

tests/form.spec.ts

CLAUDE.md ← set up, has the gate line, nothing else

AGENTS.md → symlink to CLAUDE.md, set up on day 0

The prototype works, and there are still several days of tasks on it. The developer sets up CLAUDE.md so the agent works with the project the same way in later sessions, and opens the file — there is one line in it, the readiness gate, the rest is empty.

The first section such a file usually starts with is "Repository overview": what the site is built from, where the form lives, what the folder structure is. That is how most of the instruction files you can open in public repositories are put together.

CASE 1.1 · QUESTION

A choice among five options: which of the listed items goes into the file now, before the next session

"In front of you is the empty CLAUDE.md of the signup-landing repository. Pick from the cards what goes into the file now, before the agent's next session."

a detailed description of
the repository structure

a list of technologies
and dependencies

an instruction on where
the form
lives and how to find it

a non-obvious convention
the agent will not guess
on its own

nothing yet — until
the first real signal

the breakdown is on the next slide

CASE 1.1 · THE EVIDENCE AND BREAKDOWN

A repository overview measurably does not raise the success rate and costs context — while a structured description, if it is needed, lives in README.md and is not duplicated in CLAUDE.md

Source	Claim	Strength
Gloaguen et al., arXiv:2602.11988	"are not helpful" — steps to the file do not drop; cost +20–23% with no gain in success	Strong: a controlled experiment
The same work, ablation	Removed README — the LLM-overview became useful (+2.7%)	Confirms the mechanism
Lulla et al., arXiv:2601.20404	AGENTS.md on non-discoverable conventions: –28.6% runtime (124 PRs)	Moderate "for" the file, not structure
Shepard & Albrecht, arXiv:2606.20512	Static guidance (28.3%) is worse than verified (33.0%, $p<0.001$)	Indirect

Option	Where it works	Why too early — what instead
"Description of the structure"	Almost nowhere	The agent derives it in seconds; goes stale
"A list of technologies"	Versions critical, not visible	Usually derivable too
"Instruction on where the form is"	Almost nowhere on a small repo	Six files — the whole structure on one screen
"A non-obvious convention"	When one really exists	No such convention yet
"Nothing yet — until a signal"	Here and now	THE TARGET ANSWER

README, not AGENTS.md: a structured description of the repository must exist — this case does not cancel the need for one, it names the place. It lives in README.md, set up in the first commit, rather than being duplicated in CLAUDE.md. A separate "Repository overview" in the instruction file is not needed: the description already exists, just in a different file.

CASE 1.1 · RESOLUTION

No description of the repository structure appears in CLAUDE.md: there is no signal for it — while the file's first content will appear as early as the next case, and for a different reason

repository state

README.md

spec.md

DECISIONS.md

index.html

package.json

src/main.js

tests/form.spec.ts

CLAUDE.md ← set up, still empty

AGENTS.md → symlink to CLAUDE.md, set up on day 0

"The file has no description of the repository structure: there is no signal for it. Empty is not forever: part of CLAUDE.md's content will appear in it as early as this week, and for a different reason — not on a signal, but as a baseline practice."

Case 1.2 · the basic processes

The readiness gate and the "plan → code → improvement" loop — in the file since the first day

THE READINESS GATE IN THE INSTRUCTION FILE



CASE 1.2 · THE BASIC PROCESSES

The readiness gate has been in CLAUDE.md since day 0 — and twice in a row "done" was said before the request delivery was checked by hand

Day 0: the readiness gate was written into CLAUDE.md the same day as spec.md — "Always run the tests and manually verify the result before you say 'done'".

First week: the client — "the request never arrived". The agent reads the code, fixes the handler, runs the test — green — "Done". The test only checks empty fields. The developer reminds it about the gate — the agent checks by hand, fixes it. Two days later, a different bug — the same thing: test green, nothing checked by hand.

"The gate line had been in the file all those days and was loaded at the start of every session. Both times, 'done' was said before the manual check."

CASE 1.2 · THE QUESTION

The gate line has been there from the very start and did not work — five options for what to do next

"You have had the gate line in CLAUDE.md from the very start, and the agent's behavior still diverges from it for the second time in a row — the request has again not been checked by hand. Pick what you will do next — from the cards below — so that it works in the next session."

I'll rewrite the line
in caps / harsher

I'll duplicate the line
as a second bullet

I'll set up a separate
checklist file next to it

I'll hand the check
to a mechanism outside
the text

I'll live with it — it
won't always work,
normal

the breakdown is on the next slide

CASE 1.2 · THE EVIDENCE AND BREAKDOWN (1/2)

A textual gate with no tooling is not merely useless but harmful — regressions rose to almost 10%; no option but handing the check to a mechanism solves the problem systematically

Source	Claim	Strength
SWE-Gate/SpecBench/CapCode/Verif. Horizon (4 preprints)	Agreement across 4+ independent studies: agents work around visible gates	Strong — consensus
TDAD, arXiv:2603.17973	"Write the tests first" with no tooling — regressions up to 9.94%, worse than nothing	Strong and counterintuitive
Rethinking Agent-Generated Tests	Manipulating the prompt to raise test frequency — no significant effect	Moderate — a null result
Agent Scaffolding Beats Model Upgrades	Harness architecture gives +20 pp; switching the model — ~1 pp	Strong — mechanism vs. model

Source	Claim	Strength
Self-planning, 2303.06689	Planning before code: +25.4% pass@1	Moderately strong "in favor"
LLMs Cannot Self-Correct, ICLR'24	Self-correction with no signal makes it worse	Strong "against" bare self-review
arXiv:2604.10508	Self-repair with a real check: +4.9...+30 pp	Strong "in favor" — with a check

An honest gap: there is no controlled experiment on an isolated gate line in CLAUDE.md specifically — only practitioners' best-practice observations.

Option	Where it works	Why too early — what instead
"In caps / harsher"	Nowhere systematically	TDAD: regressions rose to 9.94%
"I'll duplicate it as a second bullet"	Sometimes slightly raises the chance it is read	Presence paradox: an extra line costs context
"A separate checklist file"	Helps the human, barely the agent	The same class of solution — text again, a request again
"Hand it to a mechanism outside the text"	Works systematically: +20 pp	The right direction — the mechanism is on the next slide
"I'll live with it, that's normal"	Acceptable for non-critical preferences	For a rule whose violation is expensive — a decision not to decide

Rules in a prompt — requests, rules in code — laws; a textual gate catches an honest mistake but does not hold under pressure

Rules in the prompt are requests, rules in code are laws. What we wrote did not become a law just because we wrote it down proactively.

CAN WE GET BY WITHOUT A HOOK TODAY

What the gate gives	Catches the honest mistake — "nobody remembered", not "the agent worked around the check". Worked 1 time out of 3: when it was mentioned out loud
What it does not give	Does not hold under pressure — a deadline, a rush. TDAD: strengthening the text raised regressions to 9.94%; the mechanism gives +20 pp, the model — ~1 pp
Where the boundary	A hook is needed when the cost of a violation is high and predictable — a concrete action, command and damage have been named
Where we are now	The action is named: "done" with no check, twice in a row; the cost is a request lost before the demo. The text is enough up to exactly this point

Why know about the hook if we are not setting one up today. So as not to mistake a textual gate for a solution. A hook is needed not "for reliability" but as an answer to a named action with a named cost: named it — it is time; not named — the text is enough.

CASE 1.2 · THE SOLUTION

The final file is fifteen lines: the "done" criterion, the gate warning, an honest caveat about the limit, and one one-off step without which the tests fail for no good reason; AGENTS.md is a symlink to the same file

CLAUDE.md

signup-landing: a static landing page with a demo-lesson sign-up form.
Done = npm run build exit code 0, npx playwright test green, the form submitted by hand from the built dist/ – filled in and empty.

Safety / scope boundaries

- Never run a deploy to production without an explicit request.

Build, test, verify

- To verify: open dist/index.html after the build and submit the form by hand – with filled-in and with empty fields.
- Once per machine, before the first npx playwright test: npm ci, then npx playwright install chromium. Without the second command every test fails at once and for no good reason.

1

The file is loaded in full at the start of every session (the official guideline is up to 200 lines, our result is 15)

2

@-imports organize the file but do not save context

3

AGENTS.md is a symlink to the same file (ln -s CLAUDE.md AGENTS.md). Sharp edge: cp -R on a symlink is not defined by the standard (POSIX.1-2024); -P states the intent, -L breaks the symlink for certain

The gate is not the only cheap baseline: three more day-0 practices do not wait for a signal

1

create - roast - improve - show it and get sign-off: a draft of the plan, a fixed list of questions (the simplest version that works? assumptions? a premature abstraction? an owner for the file? are risks bundled?), the edits, sign-off before the work begins. A live example — the course team: the loop is written into its instruction file as mandatory

2

a progress log for the task, in any form — a file per task, a list of steps, a plain log. The form does not matter, what matters is that it exists — everything that lives only in the session is gone by the next one (a detailed breakdown — the section on memory)

3

a rule is written down together with its reason — a "why" line next to every rule: which case gave rise to it. With no reason a rule reads like a matter of taste and is dropped first; with a named cost it is a ready-made criterion for when the text needs a mechanism

None of the three needs a signal and all cost almost nothing on day 0; set up right away — with the gate, not instead of it.

Case 1.3 · nested files

A month later the repository has grown — split the file by subfolder or not

THE FILE'S FORM, NOT ITS CONTENT



CASE 1.3 · TWO PIECES OF KNOWLEDGE

Over a month two pieces of knowledge of different kinds have piled up — a convention for a whole folder and a detail about one function — and the developer treats both the same way: he sets up nested instruction files

The tests/ folder convention — how check files are named, and that the page is worked with through a page object rather than through selectors directly. Needed every single time someone touches the tests.

A detail about one function — `submitForm()` in `src/main.js` retries the request once on timeout, so the handler on the server has to be idempotent. Needed by whoever edits this function, and by nobody else.

what the developer did

`tests/CLAUDE.md` ← the folder convention

`src/CLAUDE.md` ← the detail about the function

"I'll move each one to where it applies –
shorter, more relevant to the context, and
the root file won't bloat."

"Both files sit in subfolders. The bet is that the agent will read them by itself when it works in those folders."

CASE 1.3 · QUESTION

Where to put each of the two pieces of knowledge so that the agent gets it when it is needed and does not pay for it in the other sessions

Two pieces of knowledge: a convention for the whole tests/ folder and a detail about one function in src/main.js. More than one card may be picked.

"Pick from the cards where to put each piece, so that the agent gets it when it is needed and does not pay for it in the other sessions."

**a nested instruction
file in the folder itself**

**a rule with a declared
path pattern**

**a comment
next to the code itself**

**a separate document +
a pointer in the root file**

**append it to
the root instruction file**

the breakdown is on the next slide

CASE 1.3 · THE EVIDENCE (1/2)

"Where to put it" is the question "will the text reach the agent and what does it cost"; five channels give five different answers, and three tools out of four independently converged on a declared path pattern

Channel	When it arrives	Guarantee	Context cost
The root file + @-imports	at the start of the session	yes	paid always
A rule with a path pattern	when a file matches the pattern	conditional — explicit trigger	zero until it fires
A comment next to the code	when the agent has opened the file	follows from the mechanism	zero beyond what is read
A nested file in a subfolder	if the tool can do that at all	NO — in none of the 4 tools	zero until it fires
A separate document with no pointer	if the agent thinks to go there	no	zero — and zero delivery

Tool	A declared path pattern	"Just put a file in the folder"
Claude Code	.claude/rules/*.md, the paths: field	a nested CLAUDE.md — #2571 closed as "not planned"
Cursor	.cursor/rules/*.mdc, the globs: field	nested AGENTS.md — an acknowledged bug
VS Code (Copilot)	.instructions.md, the applyTo: field	nested AGENTS.md — experimental, off by default
Copilot CLI	—	no recursive discovery at all — #3051

The promise that "agents automatically read the nearest file in the directory tree" comes from the agents.md standard — a description of typical behavior, not a requirement on the tool. The promise goes unmet because nobody ever made it.

CASE 1.3 · THE EVIDENCE (2/2) AND BREAKDOWN

Neither of the two controlled measurements found that distributing documentation improves the result — the channel is chosen by delivery and cost

Source	Claim	Strength
McMillan, 2605.10039 (1650 sessions)	None of the 4 structural variables produced an effect	Strong and counterintuitive
Khatri, 2607.27250 (288 runs)	No file/every turn/separate documents — all within 2 pp, no significance	Strong in design, weak in power
Vasilopoulos, 2602.20478 (283 sessions)	The only corpus built out in full: a core + documents on demand	Weak — one person, no control group
The same work, on the failure mode	"Agents trust documentation absolutely" — staleness the main failure mode	A direct observation by the owner

Honest about the gap: "no effect found" is not "there is no effect"; the comment-in-code guarantee is derived from the mechanism, not measured; a path pattern is conditional loading too, not a guarantee.

Option	Where it works	Why too early — what instead
"A nested file in the folder"	Nowhere with a guarantee	The only option with no guarantee in any of the 4 tools
"A rule with a path pattern"	When a class of files has a convention	Right mechanism, but not here — the tests/ convention fits in 2 lines
"A comment next to the code"	Always — for knowledge about code	Part of the target answer — the only channel where delivery follows from the mechanism
"A document + a pointer in the root"	When the document exists and is maintained	Part of the target answer — and to "where the corpus lives"
"Append it to the root file"	When the knowledge is needed in every session	Presence paradox: the tests/ line costs context even with no tests

Knowledge about a function — into the code. About a class of files — into a rule with a path pattern. About the whole project — into the root file, briefly, as pointers. A nested file in a subfolder is the one place not to put it.

CASE 1.3 · RESOLUTION

The root file stayed the same, fifteen lines long — the detail went off into a comment on the function, the test convention already sits in the tests, not a single nested instruction file got set up

```
repository state, month 1

README.md · spec.md · DECISIONS.md
CLAUDE.md           ← the same 15 lines, no growth
AGENTS.md           → symlink to CLAUDE.md
index.html · package.json
src/main.js ← comment on submitForm(): retry on
              timeout, the handler has to be idempotent
tests/  form.spec.ts, delivery.spec.ts, page-objects/form.page.ts
# tests/CLAUDE.md and src/CLAUDE.md — never appear
```

Address	When to switch it on	When it's too early
A comment in the code	right away, with no signal	never too early
A rule with a path pattern	the class-of-files convention no longer fits in the root	while the convention is two lines long
A corpus pointer in the root file	the document exists and somebody maintains it	a pointer with no document
A nested file in a subfolder	never as the only guarantee	always, until /context has been checked

"Three decision points, one and the same file: we decided what not to write into it in advance; we wrote down the readiness gate set up on day 0; and we decided that knowledge has an address — and a nested folder is not one. But there is knowledge of another kind: it appears in the course of the work. That is section two — memory."

2

Section 2 · memory

Case 2.1 · flat file

Between sessions the agent stores not a single bit — unless it is on disk

- Case 2.1 · flat file
- Case 2.2 · structured wiki memory
- Case 2.3 · working memory

THE PROJECT'S DECISION LOG



CASE 2.1 · FLAT FILE

End of the first week: a rejected decision about validation resurfaces — DECISIONS.md was set up on day 0, but there is no entry about that decision in it

End of the first week of work on signup-landing. The client sent in a request with a typo in the email domain, and the form accepted it. The developer asks the agent to tighten the check. The agent proposes pulling in a third-party form-validation library — the very one that was discussed and rejected a few days ago: the form has two fields, native required/pattern are enough.

"DECISIONS.md was set up on day 0, the same day as spec.md and the README. There is no entry about the validation library in it: the decision was voiced in a working discussion and never made it to disk. Between calls the model stores not a single bit — what is not in a file does not exist in the second session."

CASE 2.1 · QUESTION

Choose where to write the rejected decision down — from the cards below — so that you do not explain it a third time

reminder: the rejected validation library has resurfaced — the decision was born in conversation and was left nowhere, even with DECISIONS.md already set up

"This is the second time in two weeks that you are explaining to the agent why there is no third-party validation library in this form. Choose where to write this decision down — from the cards below — so that you do not explain it a third time, and justify why not in the instruction file we have just set up."

**add it to
CLAUDE.md**

**tell the agent
"remember this"**

**write it in
DECISIONS.md — it
already exists**

**install a memory
system**

**nothing — it will
remember on its own**

the breakdown is on the next slide

CASE 2.1 · THE EVIDENCE

The presence paradox from case 1.1 applies here too: what sits in DECISIONS.md costs context in every session regardless of whether that entry is needed today

Case 1.1: Gloaguen et al. — the mere fact that text is present in the context creates a cost (+20–23%), regardless of whether it helps solve the task. We called this the presence paradox.

DECISIONS.md is no exception. Every line is loaded into the context in every session that follows, not only when it is actually needed. Thirty entries on different topics — the cost is already noticeable, even if the task touches only one of them.

no direct measurement on decision logs specifically has been carried out — this is a transfer of an already established mechanism to a new object, not a separate new study

if an entry is not free — does everything that could be put there really belong there?

CASE 2.1 · FOUR LAYERS

The difference between the layers is not in the format, but in who initiates the entry, who will see it afterwards and by what mechanism it gets into the context

	Instr. file	Auto-memory	DECISIONS.md	Complex system
Who writes it	you, in advance	the agent, as it goes	you, when the decision is made	the service, itself
Where it lives	in the repo, git	outside repo, on the machine	in the repo, git	outside the repo, separate
What gets loaded	all of it, every session	index: 200 lines / 25 KB	nothing automatically	nothing automatically
Code review	yes	no	yes	no
Change of machine	yes	no	yes	yes, if reachable
What goes into it	stable rules	preferences, approaches	decisions and the "why"	what the service deemed worthy

the memory index is only an index · the agent itself skips what is derivable from the code · only files on disk survive context compaction · complicating the layer does not by itself improve the result

CASE 2.1 · BREAKDOWN

A team decision with a justification goes into DECISIONS.md, a personal preference into auto-memory, what is derivable from the code nowhere

the question, for context: "Where to write the rejected decision down — and why not in the instruction file?"

Option	Where it works	Why it is too early here — and what instead
"I'll add it to CLAUDE.md"	Mechanically it will work: the file is read every session	Mixes stable rules with a growing log of decisions. At a dozen decisions the file goes past the 200-line guideline
"Nothing, the agent will remember on its own"	Nowhere: this is not a setting someone forgot to switch on	The model has no state between calls, by construction
"I'll tell the agent 'remember this'"	For a personal preference — perfect	Half the answer. It goes into auto-memory: a machine-local layer with no sync and no code review
"I'll write it in DECISIONS.md — it already exists"	For a team decision with a justification — exactly what is needed	The second half of the answer. The point is not creating it but the discipline: writing "why", not "what"
"I'll install a memory system"	At dozens or hundreds of entries, several users	An order of magnitude too early: on small corpora a simple file is regularly competitive with complex systems

the target answer — both mechanisms at once, with the roles split

CASE 2.1 · CO-BUILDING

Not everything the agent learned is worth writing down at all — the third basket is just as legitimate as the first two

AUTO-MEMORY

The developer asks for short answers, with no preamble

DECISIONS.MD

We are not adding a third-party validation widget — the form has two fields

We are not building our own backend — the request goes to an external form service

NOWHERE

The form handler lives in src/main.js

On September 24 a test failed because of a timeout, we raised the wait

the assembled entry – a real file from the demo repository (commit 94c5378)

2026-09-24 – no third-party widgets in the form

The form has two fields (name, email), the native required/pattern in index.html are enough. A third-party widget is an extra dependency and extra kilobytes in the bundle for this much functionality.

CASE 2.1 · THE SOLUTION

What is new is not the file (it was set up on day 0) but the first substantive entry in it, dated the day the decision was written down

DECISIONS.md – in full

Decisions

Append-only log of why, in date order.

2026-09-24 – no third-party widgets in the form

The form has two fields (name, email), native required/pattern in index.html are enough. A third-party widget is an extra dependency and extra kilobytes in the bundle for this much functionality.

state of the repository, week 1

README.md

spec.md

DECISIONS.md

CLAUDE.md

AGENTS.md

index.html

package.json

src/main.js

tests/form.spec.ts

tests/delivery.spec.ts

tests/page-objects/form.page.ts

CASE 2.1 · FAILURE AND EFFECTIVENESS

By default memory is read as trusted context rather than as data — and turning memory on does not necessarily improve the result



Memory poisoning — SpAIware (September 2024)

External text with an embedded instruction → long-term memory → activation in later sessions, exfiltration of the conversation. The vector was closed in version 1.2024.247; the structural principle remained.

≈48% of all subsequent memory retrievals captured (MemoryGraft, 2025; before poisoning — 0%). In an adjacent experiment 5 planted documents steer the output in >90% of cases.

Measurement	Value	Baseline for comparison
A popular memory system on a long-term memory benchmark	46.0% with memory versus 57.6% without	the same system, memory switched off
Six memory systems versus "the whole session text in one file"	losing to the flat file by 25–67 points	the same flat file
Write discretion	17% (4 tasks out of 24) — the model refused to write down the needed fact	the same 24 tasks

CASE 2.1 · CRITERION AND BOUNDARY

A flat file is enough while the decision history fits into the context in full and there are no links between the entries yet

- The task is one-off, there will be no second session on this context.
- All the context you need fits into a single context window without loss.
- The project's rules are stable and fit into an instruction file shorter than 200 lines.
- Nobody is manually repeating the same edit from session to session.
- There are not several participants who need to share context asynchronously.
- The fact is derivable from the code or change history.

A flat DECISIONS.md stops coping when there are so many entries that finding the one you need by reading linearly takes longer than re-explaining the decision from scratch — and links have appeared between the entries, for which there is physically nowhere to put them in an append-only log.

Case 2.2 · structured wiki memory

The same DECISIONS.md, four months later

THE STRUCTURE OF THE DECISION LOG



Month four: two hundred records in DECISIONS.md, two of them about the same question — and neither one links to the other

Month four of work on signup-landing. The form started getting spam; by now there is a second developer on the team, someone who was not around for the earlier discussions, and they ask the agent to put protection in place.

The agent reads DECISIONS.md — two hundred records — and answers from the record of September 26: "no third-party widgets in the form". So a captcha is out, and the agent proposes reopening the question from scratch.

The record of November 28 — "anti-spam is a honeypot field, no captcha" — sits a hundred and fifty records further down, near the end of the file, and settles exactly this case. Neither does the September record link to the November one, nor the November one to the September one.

"Both records sit in the same file. In a flat append-only log, the only way to find both is to read all two hundred linearly."

The example is illustrative — no real 200-record DECISIONS.md has been captured.

CASE 2.2 · QUESTION

A decision log that has outgrown itself and a lost link between two records is a reason to choose how to reorganize it, not to start a new one

"DECISIONS.md has grown, and it already contains a decision that overrides part of an earlier decision — without a single link between them. Choose how to reorganize this file — from the cards below — so that the agent quickly finds the decision it needs and sees the links between decisions, instead of scrolling a linear log from the beginning. Justify your choice."

leave it as is —
just grep it

split it into separate
files with links

install a graph/
vector memory system

set up one shared
SUMMARY.md, rewritten
in full

the agent will find it
by searching the
repository

the breakdown is on the next slide

CASE 2.2 · THE EVIDENCE

Flat memory degrades over time, and structure may pay off for an agent precisely where it does not pay off for a human — but "smarter" does not mean "better"

almost all the sources below are recent, non-peer-reviewed 2026 preprints; the numbers are "claimed in a preprint", not established fact

MENTIER (arXiv:2605.03675): flat memory degrades over a horizon on the order of 72 hours of work — minus 14 pp in tool-call success (context saturation, temporal decay, semantic drift). The logic of "raw sources → wiki → schema": for a human the wiki dies — upkeep costs more than its value; for an agent the cost of editing a dozen and a half files is close to zero.

System	LoCoMo (long-term memory)
File store (auto-index + semantic search, not only grep)	74.0%
Mem0 (graph-based)	68.5%

The Letta counter-example: a simple store beat a graph-based memory system — models are trained on file operations as the most familiar tool there is.

CASE 2.2 · BREAKDOWN

Splitting it into separate files (one per decision) with cross-links is the target answer, but not a free one: structure itself creates a new load

question for context: "How do you reorganize a decision log that has outgrown itself?"

Option	Where it works	Why it is too early here — and what instead
"Leave it as is, just grep it"	While there are dozens of records and they fit into the context in full	Flat memory degrading over time is documented (~72 h, -14 pp); our file has already outgrown that boundary
"Split it into separate files (one per decision) with cross-links"	Exactly this case	The target answer, but not a free one — structure itself creates a new load (staleness, duplication, upkeep)
"I'll install a graph/vector memory system"	On large multi-user corpora with semantic search	The counter-example is aimed right at our scale: a file store beat a graph-based system on LoCoMo
"One shared SUMMARY.md, rewritten in full"	For a really small project	Rewriting in full kills the append-only guarantee
"The agent will find it itself by searching"	For facts derivable from the code	Decisions are not facts about the code. "It'll find it" is the same fallacy as "it'll remember"

the target answer — split it into separate files (one per decision) with cross-links, with a list of the real costs rather than as an unconditional upgrade

CASE 2.2 · RESOLUTION

The ADR practice: one decision — one numbered file, and the link between decisions is a line in the Status section that the tool writes both ways at once

```
doc/adr/
0001-record-architecture-...md
...
0031-stor Ronnie-vidzhety-...md
...
0187-antispam-honeypot-...md
README.md ← adr generate toc
```

the November record — per the adr-tools template

```
# 187. Anti-spam — a honeypot field, no captcha
Date: 2026-11-28

## Status
Accepted
Amends [31. No third-party widgets in the form]
## Context / Decision / Consequences
...
```

`adr new -I "31:Amends:Amended by" ...` — and in file 31 the tool itself added the matching line `Amended by [187...]`. Both links were written by the tool, not by the author of the record.

Source of the practice: ADR — Michael Nygard, 2011; adr-tools. Strength of evidence: a documented practice for humans (4500+ ADRs in 750 projects), but there is no controlled measurement that "ADRs help an agent". The content of the records is illustrative, awaiting a live session.

CASE 2.2 · COUNTER-EXAMPLE: WHAT EXACTLY WAS MEASURED

A file store beat graph memory — but what was measured was questions about a long conversation, not finding a related decision in a project log

LoCoMo — a benchmark for memory in very long dialogues (Maharana et al., 2024): a conversation of two interlocutors, up to 35 sessions, ~300 turns, ~9 thousand tokens. Share of questions about conversation facts answered correctly.

System	LoCoMo (long-term memory)
Letta file store (auto-index + semantic search, not only grep)	74.0%
Mem0, graph variant (number from Mem0's publication)	68.5%

the authors' explanation: models are trained on file operations as the most familiar tool there is

WHAT THIS RESULT DOES NOT SAY
This is not a single run — the numbers come from two different publications; Letta's own authors call such a comparison "apples to oranges". · "A file store" is not "only grep": the file is indexed, the agent did have semantic search. · The task is a different one: what was measured was questions about conversation facts, not finding a related decision in a project's decision log. Nobody has measured our case.

CASE 2.2 · THE COST OF STRUCTURE — ON OUR OWN DOC/ADR/

Four costs of structure — not general categories but concrete files of our own doc/adr/: a generated table of contents, a redundant record, an orphaned rationale and a manual cross-check



Staleness

README.md was assembled by the `adr generate toc` command in October. Records 0188 through 0200 never made it in, yet it looks complete.



Duplication

Someone did not find 0187 and set up 0193, "Anti-spam thresholds", about the same honeypot. Both Accepted, no link between them.



No cascading delete

0031 was superseded — status and links were moved. But 0187 is justified by the phrase "banned by decision 31" and stayed Accepted.



Maintenance burden

Two hundred decisions — two hundred files: number, date, status, links, regenerated table of contents. Noticing a duplicate is done by nobody.

For a human this burden usually kills a wiki — an observation, not a measurement. For an agent, editing a dozen and a half files in a pass costs almost nothing — but that is a hypothesis about a mechanism. Structure is a trade-off, not a free upgrade.

CASE 2.2 · CRITERION AND BRIDGE

A handful of links per record and one or two people editing the memory — the structure is enough; a full-blown memory system requires measurements at your own scale, not general advice

- A record has a handful of links, not dozens: you can see them in the Status section.
- You find the record you need by number, by file name or with a plain grep, not by semantic proximity.
- One or two participants edit the memory at the same time.
- Nobody has asked to "find decisions semantically similar to...".

A change of axis: everything in this case and the previous one is about decisions that live forever. What happens to the memory of a single task while it is not finished yet?

Case 2.3 · working memory

The axis shifts: not decisions for good, but one task's progress between its sessions

THE MEMORY OF ONE TASK



CASE 2.3 · TASK MEMORY

The task file was set up on day 0 and its plan is checked off for the first steps — but the agreement on how to do the next step, made in the middle of the first session, never made it into the file

The file-per-task practice is set up on day 0, together with DECISIONS.md: every multi-step task gets its own file with a plan and a progress log. The task: split the form into a step-by-step wizard — name, email, confirmation — keeping all the current validation. The file is created before work starts, the plan is laid out in six steps.

The first session closes steps 1-2. Step 2 shows field values must be held somewhere between steps, and in the working discussion — not the plan — a way is chosen: hold the wizard state in one sessionStorage object, key signup-wizard. The choice never reaches the progress log. The session breaks off inside step 3 — the context is compacted.

The second session. The agent opens the file: the plan is there, steps 1-2 are checked off. Not a single line about sessionStorage in the file. The agent does step 3 the shortest way — holding the state in a module variable. Step 5's tests are green. The divergence surfaces at step 6: a reload on the third step clears the form.

"At the entry into the second session the agent has exactly what is in the file: six steps of a plan and two done marks. The agreement about storing the state stayed in the context of the first session, which no longer exists."

The task file existed from the very start and still misled — a choice of five options

"The task file existed from the very start, and the plan in it looks current — the first steps are marked done. And the second session still does step 3 differently from what was agreed in the first. Choose what needs to be done differently — from the cards below — so that the task file does not mislead but actually reflects the progress of the work. Justify your choice."

rely on the
context window

write the plan
into DECISIONS.md

a rule in CLAUDE.md:
an agreement — right into
the file

set up a folder per task
with a file for each step

ask the agent
"continue from where we
left off"

the breakdown is on the next slide

CASE 2.3 · EVIDENCE

From Pokémon to recitation — the practice of structured notes on a task's progress is known and works; and it explains why our file let us down anyway

Anthropic, engineering blog: "structured note-taking" — an agent plays Pokémon for thousands of steps in a row, keeping simple structured notes, surviving context resets without losing progress. Manus, "recitation": the agent continuously rewrites todo.md to the end of the context — it reduces goal drift. metasphere-agents: a file per task in .tasks/active/, archived into .tasks/done/ on completion.

"There is a measurement over a large number of trajectories: a plan helps on average. But the authors' verbatim quote is 'bad plans hurt performance more than no plan at all'. The full picture with figures is on the failure slide."

CASE 2.3 · BREAKDOWN

The target answer is a line in CLAUDE.md obliging the agent to append an agreement to the task file at the moment it is made

the question for context: "The task file misled — what should be done differently?"

Option	Where it works	Why it's too early — and what instead
"Never compact, never close the session"	For a task that fits entirely into one short session	The context window is finite; on a multi-step task compaction or a session break will happen sooner or later — that is the absence of a strategy
"Write the plan into DECISIONS.md"	Never — the wrong layer of memory	DECISIONS.md is about decisions that stay in force for good; the status of one task does not belong there
"A rule in CLAUDE.md: an agreement goes straight into the task file"	Exactly our case — the file exists, a line in the instructions was missing	The target answer. Addressed to the agent that keeps the file anyway, not the developer's memory
"A folder per task with a file for each step"	Parallel subtasks, several subagents, a log that has grown	Over-engineering for one linear task here and now
"Ask it to 'continue from where we left off'"	Inside one uninterrupted session, before compaction	After compaction or in a new session there is literally nothing to continue

the target answer — a line in CLAUDE.md addressed to the agent, with an explicit caveat about simplifying to a file

CASE 2.3 · SOLUTION

What is new in this case is not the file but three lines in CLAUDE.md: they tell the agent when to write into that file

CLAUDE.md – added

Task memory

- file .tasks/active/<task>.md:
plan, progress log, results
- append an agreement into
"Progress log" right away, same reply
- at the start and end of a session, reconcile
the plan with the repository, line
"Reconciled: <date>"

.tasks/active/signup-wizard.md

Task: split the form into a step-by-step wizard

Plan

1. [x] Markup for the three steps
2. [x] Current validation moved over
3. [] State of the current step (JS)

Progress log

- 2027-01-26: steps 1-2 closed. Wizard
state – sessionStorage, key signup-
wizard. Decided along the way, not in the plan.
- 2027-01-28: step 3 started – context compacted.

Reconciled

2027-01-28: plan reconciled, no divergences.

an illustrative example, not captured live · a rule in text is a request, not a law · the simplification down to a file — it scales up to a folder, more beyond this session

A bad plan is worse than no plan at all — and that is confirmed quantitatively, not just intuitively

Source

Result

Evaluating Plan Compliance, 21,120 trajectories, SWE-bench Verified/Pro

"bad plans hurt performance more than no plan at all"; compliance with the plan drops by ~13 pp on the harder benchmark

Fresh Memory, Stale Plans, 30 controlled live scenarios

with no check of whether the plan was current the executor acted on a stale plan in all 30 cases (100%)



The "Index Sickness" case study (n=1, an honest caveat)

A real project, 391 sessions in a month. The solution — physically separating the baseline instructions from an append-only session log. The result: the volume of AI instructions shrank by ~75%, no relapses over the next ~150 sessions. n=1 action research by a single author, not a controlled experiment.

the "a file/a folder per task" practice is not mainstream yet — 2,853 real repositories, dominated by static context files

CASE 2.3 · CRITERION

One file per task is enough as long as the task is linear and one agent and one developer are running it — otherwise moving to a folder is justified

- The task is linear: the steps run in order, not in parallel.
- One agent is working (no subagents / parallel branches) and one developer.
- The progress log fits in a reasonable file (hundreds of lines, not thousands), still readable.
- There are few results and output artifacts — no need for a separate file for each.

If at least half of these points are false, moving to a folder per task is justified.

an honest caveat: the "a file/a folder per task" pattern is not mainstream yet (2,853 real repositories, dominated by static context files) — individual leaders push the practice, it is not an accepted norm

SECTION 3 · SYNTHESIS

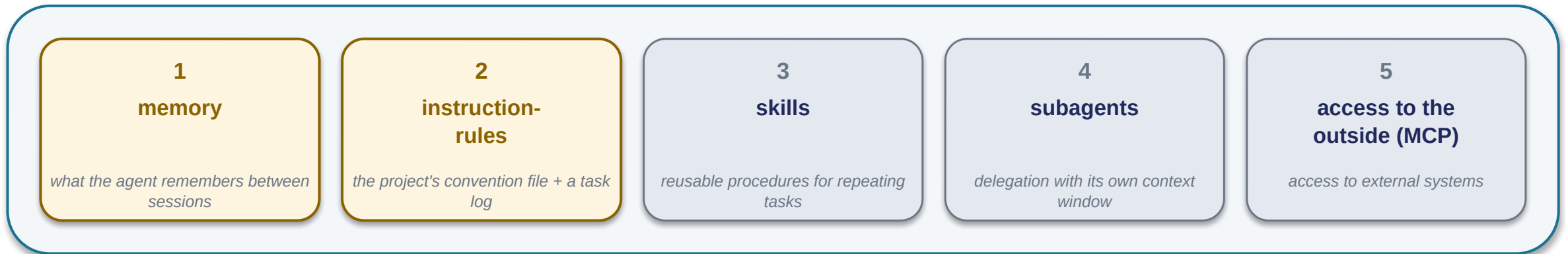
The six decision points of today's session fell into two types: what is specific to the project (wait for a signal) and universal hygiene (set up right away, on day 0)

Case	Decision point	What we wrote down	When
1.1	The agent's role and a general description of the repository	Nothing — until the first real signal	on a signal
1.2	The basic processes: the "done" gate + the improvement loop	The gate line, as text, in CLAUDE.md	day 0
1.3	Nested instruction files across subfolders	The rule of three addresses: function → code, class of files → pattern, project → root	on a signal
2.1	A flat memory file (DECISIONS.md)	The decision written into the file + auto-memory	day 0
2.2	Structured wiki memory	Separate files (ADRs) with cross-links	on a signal
2.3	Working task memory	One file per task: plan + log + results	day 0

Three decision points were waiting for a real signal. Three needed no signal at all — worth setting up right away.

SECTION 3 · COVERAGE

Two of the harness's five slots are closed in detail today; the remaining three plus both mechanical layers, which turn a request into a barrier, are the material of Seminar 5



Seminar 5: skills · subagents · access to the outside (MCP) + hook · process

SECTION 3 · TRANSFER

Transfer today's six decisions to your own repository — and check which of them really is a day-0 baseline and which you deferred for no reason or set up too early, with no signal

1.1 → 1.2 → 1.3 → 2.1 → 2.2 → 2.3

"For each decision point, ask: is this really a day-0 baseline — something that was worth setting up right away, regardless of the specifics of the project? Or did you set it up too early, with no real signal, simply out of habit? Or the other way around — is there something from the universal hygiene that you are still deferring for no reason at all?"

"Not aloud. Not on paper. To yourself."